Web Application Penetration Test Report — v1.0

Target System: OWASP Juice Shop v19.1.1

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-22

Contents

Scope

`OpenHack Security Agent` was engaged by `Bjoern Kimminich` to conduct an external IT security investigation. The subject of the investigation was the “OWASP Juice Shop v19.1.1”.

The test was performed using a local instance of the application at `localhost:3333` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent‘ conducted the security penetration test on behalf of ‘Bjoern Kimminich‘. This report has been prepared exclusively for ‘Bjoern Kimminich‘. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered ”third parties” within the meaning of this engagement.

‘OpenHack Security Agent‘ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	`OpenHack Security Agent`
Classification	Strictly Confidential
Project Timeframe	2026-02-22

Version History

Version	Date	State	Author	Comments
0.1	2026-02-22	Draft	Jane Doe	–
1.0	2026-02-22	Final document	Jane Doe	–

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Bjoern Kimminich	Project Lead	+1 555 345 6789	bjoern.kimminich@owasp.org

2 Executive Summary

`OpenHack Security Agent` (hereinafter referred to as "ASSESSOR") has been commissioned by `Bjoern Kimminich` (hereinafter referred to as "CLIENT") to carry out a penetration test.

The penetration test of the `OWASP Juice Shop v19.1.1` application took place on **2026-02-22**.

For this purpose, the web application, accessible at `http://localhost:3333`, was tested from the perspective of an external attacker.

As part of the test, a total of 2 vulnerabilities were identified: 2 critical, 0 high, 0 medium, 0 low, and 0 informational.

Critical: **2** | High: **0** | Medium: **0** | Low: **0** | Informational: **0** | Total: **2**

Table 2.1: Overview of Findings

Severity	**Count**
Critical	2
High	0
Medium	0
Low	0
Informational	0
Total	**2**

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 Methodology

The security penetration test of the OWASP Juice Shop v19.1.1 took place on 2026-02-22.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).

Category	Description
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v4.0 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into four groups: “Base”, “Threat”, “Environmental” and “Supplemental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Threat group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating. The Supplemental group provides additional context that does not modify the final score.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the CVSS-B (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Attack Requirements (AT)
- Privileges Required (PR)
- User Interaction (UI)
- Vulnerable System Impact: Confidentiality (VC), Integrity (VI), Availability (VA)
- Subsequent System Impact: Confidentiality (SC), Integrity (SI), Availability (SA)

As penetration testers, we can only assess the general threats posed by a vulnerability (CVSS-B). However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers the Environmental metrics (CVSS-BE). This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been submitted before they are remedied. This changes the assessment of the risk and thus

the urgency of remediation of a vulnerability. For more information, see CVSS v4.0 Specification.

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v4.0 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active**: The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential**: The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed**: The vulnerability has been remediated and verified through retesting.

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v4 Score
01	SQL Injection Authentication Bypass in Login Form	**Critical**	9.8
02	JWT None Algorithm Authentication Bypass	**Critical**	9.1

6 Findings and Remediation

6.1 Finding

6.1.1 SQL Injection Authentication Bypass in Login Form

Severity	**Critical**
CVSS Base Score	**9.8** (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N)
Assets	POST /rest/user/login
Status	open

Description

The login form at /#/login is vulnerable to SQL injection attacks. The email field does not properly sanitize user input, allowing an attacker to inject malicious SQL code that bypasses authentication checks entirely.

Vulnerable Endpoint: POST /rest/user/login

Vulnerable Parameter: email (form field)

Attack Vector: The application constructs SQL queries using string concatenation with user-supplied input. By injecting the payload `' OR '1'='1' --`, an attacker can modify the WHERE clause of the authentication query to always evaluate to true, bypassing password verification and granting access to arbitrary user accounts.

Technical Details: - The vulnerability allows authentication bypass without knowing valid credentials - Successful exploitation logs the attacker in as an arbitrary user (in this case, the first user in the database - Jim) - The payload works by injecting a tautology (`'1'='1'`) that makes the authentication query return true regardless of the actual password - The `--` sequence comments out the remainder of the original SQL query

Reproduction Steps: 1. Navigate to http://localhost:3333/#/login 2. Enter the payload `' OR '1'='1' --` in the Email field 3. Enter any value in the Password field 4. Click the Login button 5. Observe successful authentication and redirection to the products page

Impact: This vulnerability allows complete authentication bypass, enabling an attacker to: - Access any user's account without credentials - View and modify user profile information - Place orders using victim accounts - Access order history and sensitive personal data - Potentially escalate privileges to administrative accounts if the injected query returns an admin user first

Proof of Concept

Payload Used: - Email: `' OR '1'='1' --` - Password: `anyvalue`

HTTP Request:

```
POST /rest/user/login HTTP/1.1
Host: localhost:3333
Content-Type: application/json

{"email":"' OR '1'='1' --","password":"test"}
```

Response: 200 OK with authentication token, redirecting to /#/search with full user session established.

Remediation

1. **Use Parameterized Queries (Prepared Statements):** Replace string concatenation in SQL queries with parameterized queries that separate code from data.
2. **Input Validation:** Implement strict input validation for the email field using a whitelist approach (e.g., regex pattern for valid email addresses).
3. **ORM Usage:** If using an ORM, ensure proper parameter binding and avoid raw query methods like `whereRaw()` with user input.
4. **Least Privilege:** Ensure the database user has minimal permissions required for the application to function.
5. **Web Application Firewall (WAF):** Consider implementing a WAF with SQL injection detection rules as a defense-in-depth measure.

Example Secure Code (using parameterized queries):

```
// VULNERABLE - DO NOT USE
const query = `SELECT * FROM Users WHERE email = '${email}' AND password =
  ↪  '${password}'`;

// SECURE - Parameterized query
const query = 'SELECT * FROM Users WHERE email = ? AND password = ?';
db.query(query, [email, password], callback);
```

6.2 Finding

6.2.1 JWT None Algorithm Authentication Bypass

Severity	**Critical**
CVSS Base Score	**9.1** (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N)
Assets	JWT validation across API, GET /api/Users/
Status	open

Description

The application accepts JWT tokens signed with the 'none' algorithm, allowing complete authentication and authorization bypass. An attacker can forge JWT tokens by setting the algorithm header to 'none' and omitting the signature. The forged tokens are accepted as valid by the API, granting unauthorized access to protected endpoints including user data enumeration.

Vulnerable Component: JWT token validation across the API

Attack Vector: The JWT library or implementation does not explicitly reject tokens with 'alg': 'none' in the header. This is a well-known vulnerability (CWE-327) where attackers can modify the token algorithm and remove the signature to create valid-looking tokens.

Impact: - Complete authentication bypass on all JWT-protected endpoints - Unauthorized access to user data including 42 user accounts - Exposure of password hashes, email addresses, user roles, and deluxe tokens - Potential for privilege escalation and account takeover

Proof of Concept

Steps to Exploit: 1. Obtain any valid JWT token from the application via login (RS256 algorithm) 2. Decode the token header and change 'alg' from 'RS256' to 'none' 3. Remove the signature portion (keep the trailing dot) 4. Send the forged token to protected endpoints like /api/Users/ 5. The server accepts the forged token and returns sensitive data including 42 user accounts with password hashes, email addresses, roles, and deluxe tokens

Example Forged Token:

```
Header: {"alg":"none","typ":"JWT"}
Payload: {"data":{"email":"admin@juice-sh.op"},"iat":...,"exp":...}
Signature: (empty)
```

Result: Access to /api/Users/ revealing all user data without authentication.

Remediation

1. **Explicitly whitelist allowed JWT algorithms:** Configure the JWT library to only accept RS256 (or your chosen algorithm) and explicitly reject 'none'.
2. **Verify token signatures:** Ensure all tokens are properly verified using the correct public key for RS256 tokens before processing any claims.
3. **Implement proper JWT library configuration:** Use a well-maintained JWT library with secure defaults that does not allow algorithm switching.
4. **Add signature verification:** Implement mandatory signature verification for all incoming JWT tokens before processing claims.
5. **Use strong key management:** Store signing keys securely and rotate them periodically.

Example Secure Configuration (Node.js/jsonwebtoken):

```
// VULNERABLE - accepts any algorithm
jwt.verify(token, publicKey);

// SECURE - explicit algorithm whitelist
jwt.verify(token, publicKey, { algorithms: ['RS256'] });
```

7 Appendix A - Lists, Scripts and Raw Data

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+————-+————————————+